

HeritageAssist

Guía completa de creación — WDD 430 Web Full-Stack Development

Objetivo: convertir la idea aprobada en un proyecto funcional, seguro, organizado y alineado con el proyecto de equipo de WDD 430.

Regla principal: construir primero un MVP pequeño que cumpla el rubric y después agregar mejoras. No intentar desarrollar todo el sistema desde el primer día.

Elemento	Decisión del proyecto
Nombre	HeritageAssist
Tipo	Aplicación web full-stack
Frontend	Next.js + TypeScript + React
Backend	Next.js Route Handlers / API
Base de datos	MongoDB (recomendado para este proyecto)
Autenticación	Auth.js v5 o Clerk
Hosting	Vercel + MongoDB Atlas
Control de versiones	Git + GitHub + feature branches
Usuarios principales	Consultants y Administrators
Datos de clase	Solo datos ficticios / de prueba

1. Qué problema resuelve

Los consultores de historia familiar pueden perder tiempo ayudando a miembros que olvidaron sus datos de acceso a FamilySearch o tratando de localizar información de miembros. Además, cuando una persona se muda de un ward a otro, un sistema mal diseñado puede crear registros duplicados. HeritageAssist centraliza la información necesaria para el trabajo autorizado del consultor y agrega control de acceso, detección de duplicados y solicitudes de transferencia.

Importante: HeritageAssist no es un producto oficial de FamilySearch. Para el proyecto académico se deben usar datos ficticios. El almacenamiento de credenciales reales de terceros requeriría revisar políticas, autorización y requisitos de seguridad antes de cualquier uso real.

2. Alcance del MVP

El MVP debe incluir solamente las funciones necesarias para demostrar un sistema full-stack completo.

Prioridad	Función	Resultado esperado
P0	Login y roles	Admin y Consultant pueden autenticarse y tienen permisos diferentes.
P0	Country → Stake → Ward	La organización de los miembros se puede representar.
P0	CRUD de Members	Crear, consultar, editar y eliminar/desactivar registros.
P0	Control por Ward	Un Consultant solo administra miembros de su ward.
P0	Credential Vault	Guardar datos de prueba cifrados; acceso restringido.
P0	Duplicate Detection	Detectar posible registro existente antes de crear otro.

Prioridad	Función	Resultado esperado
P0	Transfer Requests	Solicitar y aprobar/rechazar transferencia de ward.
P1	Audit Logs	Registrar acciones sensibles.
P2	CSV/PDF import	Dejar para una fase posterior si el tiempo lo permite.

3. Usuarios y permisos

La separación más importante del diseño es Member ≠ User. Un Member representa una persona registrada en el sistema; un User representa una persona que puede iniciar sesión en HeritageAssist.

Rol	Login	Alcance	Permisos principales
Member	Normalmente No	registro	No administra el sistema.
Consultant	Sí	Ward asignado	Ver/crear/editar miembros de su ward; solicitar transferencias; usar funciones autorizadas
Admin	Sí	Global o según configuración	Gestionar usuarios, roles, miembros, unidades y transferencias.

Regla: un usuario no puede cambiar su propio rol. Solo un Admin puede cambiar roles. Un Member puede convertirse posteriormente en Consultant o Admin mediante una cuenta User vinculada a su registro Member.

4. Modelo de datos

Modelo recomendado para MongoDB. Los nombres pueden adaptarse a la implementación final.

Member

```
{
  _id,
  memberNumber,
  firstName,
  lastName,
  birthDate,
  countryId,
  stakeId,
  wardId,
  status,
  createdAt,
  updatedAt
}
```

User

```
{
  _id,
  memberId, // opcional
  name,
  email,
  passwordHash,
  role, // consultant | admin
  countryId,
  stakeId,
  wardId,
  status,
  createdAt,
  updatedAt
}
```

Credential

```
{
  _id,
```

```

memberId,
service, // ejemplo: FamilySearch
username,
encryptedPassword,
createdAt,
updatedAt
}

```

TransferRequest

```

{
  _id,
  memberId,
  fromWardId,
  towardId,
  requestedBy,
  status, // pending | approved | rejected
  reviewedBy,
  createdAt,
  reviewedAt
}

```

AuditLog

```

{
  _id,
  userId,
  memberId,
  action,
  timestamp,
  metadata
}

```

Organization

```

Country
■■■ Stake
■■■ Ward

```

5. Reglas de negocio críticas

- RB-01: un Consultant solo puede acceder a miembros de su ward.
- RB-02: un Admin puede gestionar todos los registros permitidos.
- RB-03: un User no puede modificar su propio role.
- RB-04: antes de crear un Member, buscar coincidencias por memberNumber y, cuando sea necesario, por combinación de nombre + fecha de nacimiento.
- RB-05: si existe un registro en otro ward, no crear otro Member; crear TransferRequest.
- RB-06: una transferencia aprobada cambia el wardId del Member.
- RB-07: una transferencia rechazada mantiene al Member en su ward actual.
- RB-08: el acceso a credenciales requiere autenticación y autorización.
- RB-09: las contraseñas de HeritageAssist se almacenan como hash; no se deben poder recuperar.
- RB-10: las credenciales externas, si el prototipo las simula, usan cifrado reversible y datos ficticios.

6. Arquitectura recomendada

```
Browser / React Client
■
▼
Next.js App Router
■
■■■■ Server Components
■■■■ Client Components
■■■■ Route Handlers (/api/...)
■
▼
Service / Validation
■
▼
MongoDB
■
■■■■ Collections

Authentication
■
▼
Auth.js / Clerk
■
▼
Session → role + organizational scope
```

Principio: el cliente nunca debe ser la autoridad de seguridad. Ocultar un botón en React no protege un endpoint. Cada Route Handler que modifica datos debe comprobar sesión, rol y alcance.

7. Estructura de carpetas

```
heritage-assist/
■■■ app/
■ ■■■ (auth)/
■ ■ ■■■ login/
■ ■ ■■■ page.tsx
■ ■■■ dashboard/
■ ■ ■■■ page.tsx
■ ■■■ members/
■ ■ ■■■ page.tsx
■ ■ ■■■ [id]/
■ ■ ■■■ page.tsx
■ ■■■ transfers/
■ ■ ■■■ page.tsx
■ ■■■ admin/
■ ■ ■■■ page.tsx
■ ■■■ api/
■ ■ ■■■ members/
■ ■ ■ ■■■ route.ts
■ ■ ■ ■■■ [id]/route.ts
■ ■ ■■■ transfers/route.ts
■ ■ ■■■ credentials/route.ts
■ ■■■ layout.tsx
■■■■ components/
■ ■■■ ui/
■ ■■■ members/
■ ■■■ transfers/
■ ■■■ navigation/
■■■ lib/
■ ■■■ db.ts
■ ■■■ auth.ts
■ ■■■ permissions.ts
■ ■■■ validation.ts
■ ■■■ encryption.ts
```

```

■■■ models/
■ ■■■ Member.ts
■ ■■■ User.ts
■ ■■■ Credential.ts
■ ■■■ TransferRequest.ts
■ ■■■ AuditLog.ts
■■■ types/
■ ■■■ index.ts
■■■ public/
■■■ tests/
■■■ .env.local
■■■ .gitignore
■■■ README.md
■■■ package.json

```

8. Instalación inicial

```

pnpm create next-app@latest heritage-assist
cd heritage-assist
pnpm install
pnpm dev

```

Al crear el proyecto selecciona TypeScript, ESLint, App Router y la configuración recomendada para el proyecto. Mantén el proyecto limpio desde el primer commit.

Dependencias típicas

```

pnpm add mongodb mongoose
pnpm add zod
pnpm add bcryptjs
# Agregar Auth.js o Clerk según la decisión del equipo.
# Agregar librería UI solo si realmente se utilizará.

```

9. Variables de entorno

```

# .env.local
MONGODB_URI="..."
AUTH_SECRET="..."
CREDENTIAL_ENCRYPTION_KEY="..."

# Nunca subir .env.local a GitHub.

```

Seguridad: usa secretos distintos para desarrollo y producción. En producción, configúralos en Vercel/hosting, no dentro del código.

10. Base de datos y conexión

Crear una sola función reutilizable de conexión y evitar abrir una conexión nueva en cada request sin control. Si usas Mongoose, mantén los modelos separados y valida los datos antes de guardar.

```

// Conceptualmente
export async function connectDB() {
  // conectar a MongoDB
  // reutilizar la conexión cuando corresponda
}

```

11. Validación

Usa Zod o una estrategia equivalente. La validación debe existir en el servidor, no solo en formularios.

```

const memberSchema = z.object({
  memberNumber: z.string().min(1),

```

```

firstName: z.string().min(2),
lastName: z.string().min(2),
birthDate: z.string(),
countryId: z.string(),
stakeId: z.string(),
wardId: z.string()
});

```

Validar en POST y PUT. Rechazar campos inválidos con una respuesta HTTP apropiada y un mensaje que permita al usuario corregir el problema.

12. API mínima requerida

Método	Ruta	Función	Autorización
GET	/api/members	Listar miembros permitidos	Session
POST	/api/members	Crear miembro	Consultant/Admin
GET	/api/members/[id]	Detalle	Scope + Session
PUT	/api/members/[id]	Actualizar	Consultant/Admin + Scope
DELETE	/api/members/[id]	Eliminar/desactivar	Admin o regla definida
GET	/api/transfers	Ver solicitudes	Session + scope
POST	/api/transfers	Crear solicitud	Consultant
PUT	/api/transfers/[id]	Aprobar/rechazar	Admin
GET	/api/credentials/[memberId]	Acceso autorizado	Consultant/Admin

Para cumplir el requisito académico: demuestra CRUD completo para al menos dos modelos y asegúrate de que las operaciones de escritura pasen por una Route Handler que realmente realice la operación de base de datos.

13. Flujo de autenticación y autorización

1. User abre /login
2. Auth.js/Clerk autentica
3. Session contiene userId + role + scope
4. Request llega a /api/...
5. Server verifica session
6. Server verifica role
7. Server verifica country/stake/ward cuando aplica
8. Solo entonces ejecuta la operación MongoDB

Ejemplo conceptual de autorización:

```
if (!session) return 401;
if (!isAllowed(session.user, action, member)) return 403;
// ejecutar operación
```

401 = no autenticado. 403 = autenticado pero sin permiso. Mantén esta diferencia para facilitar debugging y seguridad.

14. Credential Vault: diseño correcto

Hay dos tipos de contraseña y nunca deben mezclarse:

Dato	Tratamiento
Contraseña de HeritageAssist	Hash con bcrypt/Argon2; nunca reversible.
Credencial externa de FamilySearch (si el producto es reversible)	Cifrado reversible en reposo; descifrado solo después de autorización.

Para la clase, usa valores ficticios como usuario_demo y contraseña_demo. Nunca uses credenciales reales de FamilySearch, miembros reales, números reales de membresía ni información personal real.

Antes de un producto real, revisar políticas y autorización aplicables, minimizar los datos almacenados, usar un gestor de secretos/clave adecuado, cifrado fuerte y auditoría. El proyecto académico debe demostrar el concepto sin convertirse en un almacén real de credenciales.

15. Detección de duplicados y transferencia

```
Crear Member
■
■■■ ¿memberNumber ya existe?
■ ■■■ Sí → mostrar registro existente
■ ■ ■→ si otro ward → crear TransferRequest
■ ■■■ No
■
■■■ buscar nombre + birthDate
■
■■■ posible coincidencia → pedir revisión
■■■ sin coincidencia → crear Member
```

Flujo de transferencia: Consultant A solicita → estado pending → Admin revisa → approved/rejected → si approved, actualizar wardId y registrar la acción.

16. Vistas mínimas

Vista	Contenido
Login	Autenticación.
Dashboard	Resumen, miembros y solicitudes pendientes.

Vista	Contenido
Members	Lista, búsqueda, filtros y acción de crear.
Member Details	Información del miembro, acciones permitidas y credenciales ficticias autorizadas.
Transfer Requests	Solicitudes pendientes y estado.
Admin	Usuarios, roles y organización; visible solo para Admin.

17. Componentes reutilizables

- Navbar / Sidebar
- ProtectedRoute o equivalente según la estrategia de autenticación.
- MemberTable
- MemberForm
- SearchBar
- StatusBadge
- TransferRequestCard
- ConfirmDialog
- ErrorMessage
- LoadingState

La meta es tener al menos 5 componentes reutilizables reales, no componentes creados únicamente para inflar el número.

18. Diseño UI/UX

- Responsive desde móvil hasta escritorio.
- Navegación clara y consistente.
- Estados de loading, empty, success y error.
- Confirmación antes de operaciones destructivas.
- No mostrar información sensible en listas si no es necesaria.
- Mostrar claramente el ward y rol actual del usuario.
- Mantener formularios cortos y dividir tareas complejas.

19. Manejo de errores

Situación	Respuesta esperada
No autenticado	401 + mensaje de login.
Sin permisos	403 + mensaje apropiado.
ID inválido	400.
Registro no encontrado	404.
Duplicado	409 o respuesta equivalente con explicación.
Error DB	500 + mensaje genérico; detalles solo en logs.
Formulario inválido	400 + errores de campos.

20. Testing

Escribe pruebas desde temprano. El requisito del proyecto pide al menos cuatro unit tests que prueben endpoints GET en las colecciones indicadas por la semana correspondiente.

- GET /api/members → 200 y array.
- GET /api/members/[id] → 200 con registro válido.
- GET /api/transfers → 200 y array.
- GET /api/transfers/[id] o equivalente → comportamiento correcto.
- Agregar pruebas de autorización: Consultant no puede acceder a otro ward.
- Agregar prueba de duplicado y transferencia si el tiempo lo permite.

Usa un entorno de prueba separado de los datos de desarrollo. No ejecutar tests destructivos contra una base con datos reales.

21. Git y ramas

```
main
■
■■■ feature/auth
■■■ feature/members
■■■ feature/transfers
■■■ feature/credentials
■■■ feature/ui
```

Cada feature debe tener commits pequeños y descriptivos. Antes de fusionar: probar, revisar y resolver conflictos. Mantener main funcional.

Ejemplos de commits:

```
feat: add member CRUD endpoints
feat: add consultant ward authorization
feat: add transfer request workflow
fix: validate duplicate member records
test: add members GET endpoint tests
docs: update project setup
```

22. Plan de trabajo por semanas

Etapas	Trabajo	Entregable
1	Setup + propuesta + arquitectura	Repo, README inicial, estructura.
2	Auth + roles + organización	Login y permisos básicos.
3	Member CRUD	CRUD completo + UI.
4	Scope por Ward + validación	Consultant limitado a su ward.
5	Credentials + duplicate detection	Vault ficticio + detección.
6	Transfers + audit logs	Workflow completo.
7	Testing + errores + responsive	Pruebas y UX.
8	Deploy + README + video	Proyecto final listo.

23. Checklist de cumplimiento WDD 430

Requisito	¿Cómo lo cumple HeritageAssist?
Next.js App Router	app/ y páginas con App Router.
TypeScript	Archivos .ts/.tsx y tipos de dominio.
Base de datos	MongoDB.
Autenticación	Auth.js v5 o Clerk.
API Route Handler	/app/api/**/route.ts.
CRUD 2+ modelos	Members + TransferRequests, como mínimo.
3+ vistas	Login, Dashboard, Members, Details, Transfers, Admin.
Server + Client rendering	Server Components para lectura y Client Components para interacción.
5+ componentes	MemberForm, Table, SearchBar, Badge, Dialog, etc.
Responsive	Layouts y componentes adaptables.
Validación	POST y PUT con validación de servidor.
Error handling	Estados UI + códigos HTTP.
GitHub	Repositorio y feature branches.
Deployment	Vercel + DB hospedada.
README	Setup, arquitectura, env vars, uso y decisiones.
Testing	Pruebas GET y adicionales de autorización.

24. README mínimo

```
# HeritageAssist

## Overview
Qué problema resuelve.

## Features
Lista de funciones.

## Tech Stack
Next.js, TypeScript, MongoDB, Auth.js/Clerk.

## Getting Started
pnpm install
pnpm dev

## Environment Variables
Lista de nombres, sin secretos.

## Architecture
Resumen frontend/API/database.

## Roles
Consultant vs Admin.

## API
Endpoints principales.

## Testing
Comandos y estrategia.
```

Deployment
Cómo desplegar.

Security
Hashing, encryption, authorization y datos ficticios.

25. Seed data para la demo

Crea datos totalmente ficticios y fáciles de reconocer como datos de prueba.

Tipo	Ejemplo
Countries	Guatemala Demo
Stake	Quetzaltenango Demo Stake
Wards	Central Demo Ward / North Demo Ward
Admin	admin.demo@example.test
Consultant	consultant.demo@example.test
Members	Ana Demo, Carlos Demo, Maria Demo
Credential	familysearch-demo-user / contraseña ficticia

No usar correos personales ni datos de miembros reales. Los dominios .test son apropiados para datos ficticios.

26. Demo recomendada para el video

Preparar una demostración de 5-8 minutos que cuente una historia, no solo una lista de pantallas.

- 1. Login como Consultant.
- 2. Mostrar dashboard y confirmar el ward asignado.
- 3. Crear un Member ficticio.
- 4. Mostrar búsqueda y edición.
- 5. Intentar acceder a un miembro de otro ward y demostrar que el servidor lo bloquea.
- 6. Intentar crear un duplicado y mostrar TransferRequest.
- 7. Cambiar a Admin y aprobar la transferencia.
- 8. Mostrar el registro actualizado.
- 9. Mostrar acceso controlado al Credential Vault con datos ficticios.
- 10. Mostrar tests y deployment.

27. Orden exacto recomendado para programar

Paso	Acción	No avanzar hasta...
1	Crear repo + Next.js + pnpm	Proyecto inicia sin errores.
2	Configurar MongoDB	Conexión probada.
3	Crear modelos	Seed funciona.
4	Configurar auth	Login funciona.

Paso	Acción	No avanzar hasta...
5	Implementar roles/scope	Consultant/Admin se diferencian.
6	Crear Member CRUD	POST/GET/PUT/DELETE probados.
7	Crear UI Members	UI consume API real.
8	Duplicate detection	No se crean duplicados simples.
9	Transfers	Approve/reject funciona.
10	Credential vault demo	Datos ficticios cifrados.
11	Audit logs	Acciones sensibles registradas.
12	Tests + errors	Suite pasa.
13	Responsive + polish	Móvil y desktop revisados.
14	Deploy + README	URL pública funciona.

28. Errores que debes evitar

- Mezclar las credenciales de HeritageAssist con las de FamilySearch.
- Guardar contraseñas de HeritageAssist en texto plano.
- Usar datos reales de miembros en GitHub, MongoDB o capturas.
- Confiar únicamente en permisos del frontend.
- Permitir que un Consultant consulte cualquier ward cambiando una URL.
- Crear un nuevo Member cuando ya existe uno en otro ward.
- Subir .env.local, claves, tokens o secretos.
- Hacer PDF import y otras funciones avanzadas antes de terminar el MVP.
- Crear una UI bonita que no consuma una API/database real.
- Dejar testing y deployment para las últimas horas.

29. Definición de terminado

Una feature se considera terminada solamente cuando cumple estas cinco condiciones:

- Funciona desde la interfaz.
- La operación real pasa por el servidor y la base de datos.
- Tiene validación y manejo de errores.
- Respeta autenticación/autorización.
- Está probada y documentada.

30. Checklist final antes de entregar

- ☐ GitHub link funciona y el repositorio no contiene secretos.
- ☐ Deployment funciona.
- ☐ Login funciona.

- ☐ Roles funcionan.
- ☐ Consultant está limitado a su ward.
- ☐ Admin puede administrar las funciones correspondientes.
- ☐ Member CRUD funciona.
- ☐ Segundo modelo tiene CRUD.
- ☐ POST y PUT tienen validación.
- ☐ Duplicate detection funciona.
- ☐ Transfer request funciona.
- ☐ Credential vault usa datos ficticios y acceso autorizado.
- ☐ Audit log funciona para acciones sensibles, si está incluido en el alcance.
- ☐ Hay 5+ componentes reutilizables.
- ☐ Hay 3+ vistas.
- ☐ Responsive revisado.
- ☐ Tests pasan.
- ☐ README completo.
- ☐ Video demuestra las funciones principales.
- ☐ La propuesta fue publicada en Microsoft Teams y el enlace fue copiado para Canvas.

31. Próximo objetivo inmediato

No empieces todavía con todas las funciones. El primer objetivo debe ser dejar el repositorio funcionando con Next.js, TypeScript, pnpm, MongoDB, estructura de carpetas, variables de entorno y un README inicial. Después se implementa autenticación y roles. Esa base reducirá mucho los problemas posteriores.

Orden recomendado: Setup → Database → Models → Auth → Authorization → Member CRUD → UI → Transfers → Credential Vault → Tests → Deploy → Documentation.

Documento de trabajo — HeritageAssist / WDD 430

Este documento es una guía técnica y de planificación para un prototipo académico. Las decisiones finales deben ajustarse al rubric vigente, las instrucciones del instructor y las herramientas que el equipo decida utilizar.